> All the best for today's **Naan Mudhalvan Practical Exam** bro! பயப்படவே வேணாம் — உங்க **ComicCraft** project ரொம்ப neat-ஆ, strong features-ஓட இருக்கு. உங்க repo-ல இருக்குற code (`Coding/app/`) மற்றும் official project report (`07_Final_Project_Report.docx`) எல்லாத்தையும் முழுசா பார்த்துட்டேன்.
>
> Exam paper-ல அப்படியே பார்த்து எழுதுற மாதிரி **English Content** + Viva-ல மாஸ் காட்ட **Tanglish** புரியற மாதிரி **Explanation & Q&A** கீழே குடுத்திருக்கேன்.

1. முதல்ல **Project** என்ன பண்ணுது? (1-Minute Tanglish Summary)

ComicCraft அப்படின்றது ஒரு **AI-Powered Comic Book Creator Web Application** (Python **FastAPI** வச்சு பண்ணது).

1. User ஒரு சின்ன **Story Idea** (உதாரணத்துக்கு: "A brave fox follows glowing leaves to a hidden city"), **Character Name**, **Setting**, **Tone**, **Art Style**, மற்றும் **Panel Count (3 to 8 panels)** குடுப்பாங்க.
2. நம்ம ஆப் **Google Gemini AI**-ய **2 Stages**-ஆ யூஸ் பண்ணும்:
   - **Stage 1 (Outline Model):** கதைக்கு Title, **Character Bible** (character பாக்க எப்படி இருப்பாங்கன்னு நிலையான description), ஒவ்வொரு panel-க்கும் scene plan ரெடி பண்ணும்.
   - **Stage 2 (Story Model):** அந்த outline-ஐ வச்சு ஒவ்வொரு panel-க்கும் **Caption, Narration, Dialogue, Image Prompt** எழுதும்.
3. எல்லா AI output-உம் சும்மா text-ஆ இல்லாம **Strict JSON + Pydantic Validation** வழியா check ஆகும்.
4. அடுத்து **Image Service** ஒவ்வொரு panel-க்கும் ஒரு PNG படம் உருவாக்கும் (**Hugging Face SDXL** / **Local Stable Diffusion** / API key இல்லனா **Pillow Placeholder Art**).
5. கடைசியா `fpdf2` வச்சு Cover Page + Panel Pages ஓட ஒரு அழகான **A4 PDF Comic Book** உருவாக்கி, browser-ல preview காமிச்சு Download பண்ண குடுக்கும்!
6. சூப்பர் பிளஸ் பாயிண்ட் **(Fallback Design):** API key இல்லனாலும் / Internet இல்லனாலும் ஆப் crash ஆகாது! உள்ளேயே இருக்குற **Offline Demo Writer + Placeholder Image** வச்சு கம்ப்ளீட்டா ஓர்க் ஆகும்.

2. Exam Paper-ல எழுத வேண்டிய **Content (Ready to Write)**

Practical exam sheet-ல இந்த headings போட்டு எழுதுங்க — பாக்க ரொம்ப professional-ஆ இருக்கும், full marks கிடைக்கும்:

PROJECT TITLE

ComicCraft – AI Comic Story Creator Using Gemini Models

AIM

To design and develop **ComicCraft**, an AI-assisted web application using **Python and FastAPI** that automatically converts a user's short story idea into a structured, multi-panel illustrated comic book with browser preview and downloadable **A4 PDF** export, along with resilient offline fallback support.

ABSTRACT (Exam-க்கு ஈஸியா மனப்பாடம் பண்ணி எழுத — Crisp Version)

> **ComicCraft** is an AI-powered web application built with **FastAPI** that transforms a short story idea into a complete multi-panel comic book. The user enters a creative brief—story idea, main character, setting, tone, art style, and panel count (3 to 8 panels)—through a responsive web interface or RESTful JSON API. The system uses a **two-stage Google Gemini AI workflow**: the first stage generates a structured JSON story outline along with a Character Bible for visual consistency, and the second stage expands it into a complete comic script containing panel captions, narration, dialogue, and image prompts. All inputs and AI responses are strictly validated using **Pydantic v2** schemas. Panel illustrations are generated through a pluggable image pipeline supporting **Hugging Face Inference (SDXL)**, **Local Stable Diffusion (Diffusers)**, and a credential-free **Pillow Placeholder** generator. Finally, the artwork and script are compiled into a Unicode-supported **A4 PDF** using **fpdf2** and stored in a UUID-isolated local workspace (`comic.json`, PNG panels, and `comic.pdf`). To ensure high reliability, the system includes exponential-backoff retries, model fallbacks, and an offline deterministic demo mode so the application works seamlessly even without API keys or internet access.

(Keywords எழுத சொன்னா இத போடுங்க):
Keywords: FastAPI, Google Gemini API, Generative AI, Pydantic Validation, Text-to-Image, Stable Diffusion, Hugging Face, fpdf2, Fallback Architecture.

EXISTING SYSTEM vs PROPOSED SYSTEM

Existing System:

- Manual comic creation requires multiple separate skills: story writing, panel scripting, drawing illustrations, and page layout design.
- Naive AI chat tools generate unstructured plain text, which breaks automated pipelines.
- Character appearance changes randomly from one image panel to another (lack of visual continuity).
- API rate limits or missing credentials cause complete application failure.

Proposed System:

- Single integrated web platform from story brief to downloadable A4 PDF comic.
- **Two-Stage Gemini JSON Workflow** strictly validated with **Pydantic** contracts (no brittle string splitting).
- Uses a **Character Bible** repeated across prompts to maintain character consistency across panels.
- **Graceful Fallback Mechanism** (Offline Demo Writer + Pillow Placeholder Art) ensures 100% uptime and offline testability without paid API keys.

SYSTEM REQUIREMENTS

Software Requirements:

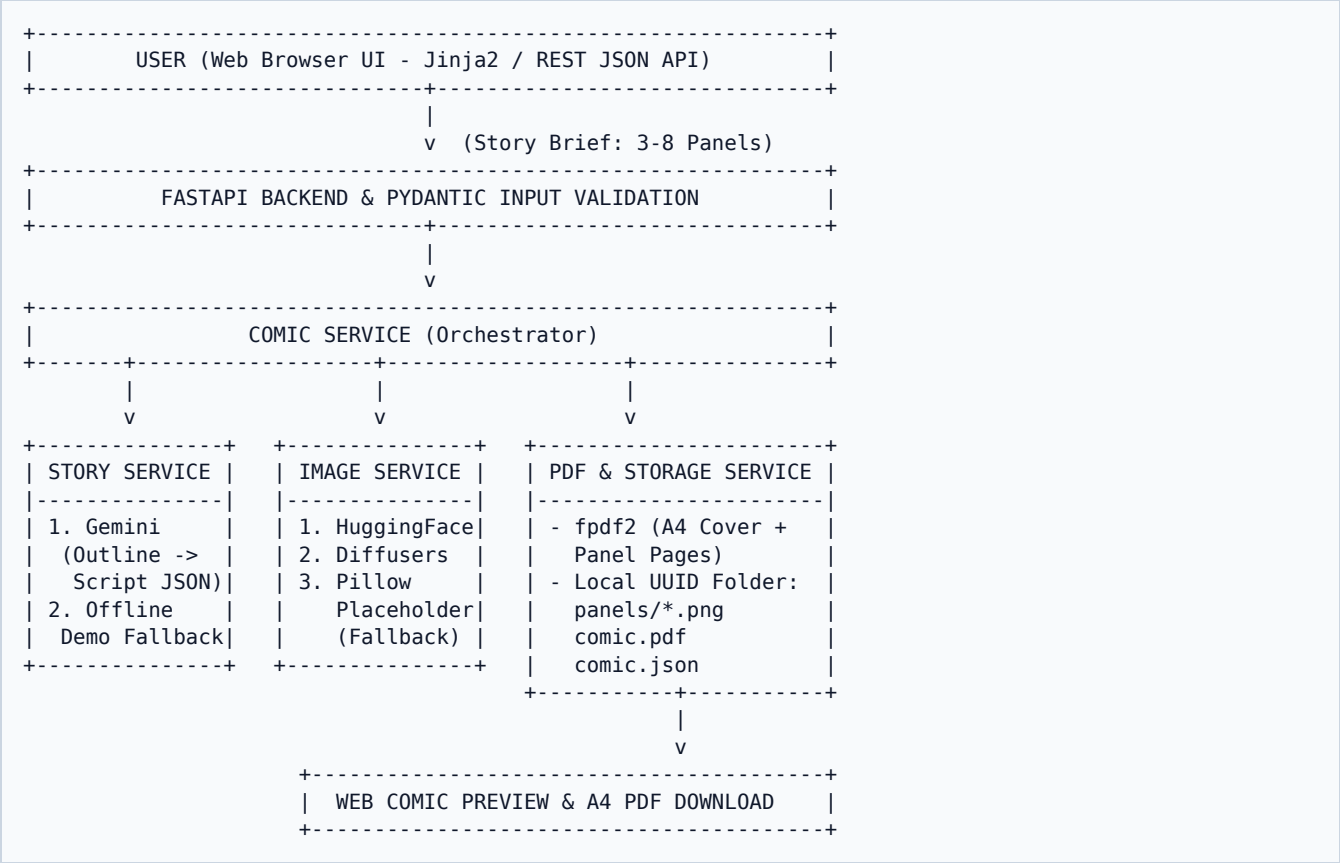
- **Programming Language:** Python 3.11+
- **Backend Framework & Server:** FastAPI, Uvicorn (ASGI Server)
- **Data Validation & Config:** Pydantic v2, `pydantic-settings`
- **AI Story Generation:** Google Gemini (`google-genai` SDK) + Offline Demo Writer
- **Image Generation:** Pillow (Placeholder), Hugging Face Hub (SDXL), PyTorch/Diffusers (Optional Local Stable Diffusion)
- **PDF Generation:** `fpdf2` with bundled DejaVu Sans Unicode TTF fonts
- **Frontend:** Jinja2 Templates, HTML5, CSS3, Vanilla JavaScript

- **Testing:** `pytest`, `pytest-cov`, FastAPI `TestClient`

Hardware Requirements:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher
- **RAM:** 4 GB minimum (8 GB recommended)
- **Storage:** ~250 MB for Python environment + local storage for generated comics
- **GPU:** Not required for Demo / Hugging Face modes (CUDA GPU optional only for Local Stable Diffusion)

SYSTEM ARCHITECTURE (Pen & Scale வச்சு பேப்பர்ல வரைய ஈஸியான Diagram)



ALGORITHM / WORKING PROCEDURE (Step-by-Step)

- **Step 1 (Input Collection):** Start the FastAPI server and accept user inputs: `story_prompt` (10–1500 chars), `character_name`, `setting`, `tone`, `art_style`, and `panel_count` (3–8, default 5).
- **Step 2 (Input Validation):** Validate inputs using the `ComicPrompt` Pydantic schema and create an isolated UUID workspace under `storage/comics/<comic_id>/`.
- **Step 3 (Two-Stage Story Generation):**
  - Stage 1 (Outline): Call Gemini Outline Model (`gemini-3.5-flash-lite`) in JSON mode to generate comic title, `character_bible`, and panel scene outlines (`ComicOutline`).
  - Stage 2 (Script): Call Gemini Story Model (`gemini-3.5-flash`) to generate panel captions, narration, dialogue (up to 3 lines), and detailed image prompts (`ComicScript`).
  - Fallback: If API is unavailable or in `demo` mode, generate script via deterministic `DemoStoryProvider`.
- **Step 4 (Panel Artwork Generation):** For each panel, compute a deterministic SHA-256 seed and generate `panel-N.png` using the configured provider (`placeholder`, `huggingface`, or `diffusers`).
- **Step 5 (PDF Compilation & Persistence):** Build an A4 PDF (`comic.pdf`) with a styled cover page and per-panel pages using `fpdf2`, and atomically save metadata to `comic.json`.

- **Step 6 (Preview & Download):** Render the comic panels and provider status in `comic_preview.html` and serve the downloadable PDF via `/download/{comic_id}`.

SAMPLE CORE CODE (பேப்பர்ல Code எழுத சொன்னா இந்த சுருக்கமான Core Pipeline எழுதுங்க)

```
# File: app/routes.py & app/services/comic_service.py
from fastapi import APIRouter, Form, Request
from fastapi.concurrency import run_in_threadpool
from app.schemas import ComicPrompt

router = APIRouter()

@router.post("/generate")
async def generate_web(
    request: Request,
    story_prompt: str = Form(min_length=10, max_length=1500),
    character_name: str = Form(min_length=1, max_length=80),
    setting: str = Form(min_length=1, max_length=100),
    tone: str = Form(min_length=1, max_length=50),
    art_style: str = Form(min_length=1, max_length=80),
    panel_count: int = Form(ge=3, le=8, default=5),
):
    payload = ComicPrompt(
        story_prompt=story_prompt,
        character_name=character_name,
        setting=setting,
        tone=tone,
        art_style=art_style,
        panel_count=panel_count,
    )
    # Run synchronous AI, Image, and PDF pipeline in threadpool
    comic = await run_in_threadpool(comic_service.generate, payload)
    return templates.TemplateResponse("comic_preview.html", {"request": request, "comic": comic})
```

```
# Core Orchestration inside ComicService.generate(prompt)
def generate(self, prompt: ComicPrompt) -> ComicResult:
    comic_id, workspace = self.repository.create_workspace()
    story = self.story_service.generate(prompt)  # Two-stage Gemini / Demo fallback

    image_paths = []
    for panel in story.script.panels:
        dest = workspace / "panels" / f"panel-{panel.panel_number}.png"
        res = self.image_service.generate_panel(panel, dest, comic_id)
        image_paths.append(res.path)

    pdf_path = workspace / "comic.pdf"
    self.pdf_service.build(story.script, prompt, image_paths, pdf_path)
    self.repository.save(result)  # Atomically writes comic.json
    return result
```

RESULT

Thus, the AI-assisted comic creation web application **ComicCraft** was successfully developed using FastAPI, Google Gemini structured JSON workflow, pluggable image generation, and `fpdf2` PDF export, and verified across both web and API interfaces.

3. Lab-ல Run பண்ணி காட்ட சொன்னா (Execution Steps)

API Key எதுவுமே இல்லாம Offline Demo Mode-ல 10 செகண்ட்ல run பண்ணலாம்:

```
cd Coding
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

- **Home Page (Form):** `http://127.0.0.1:8000/`
- **Health Check JSON:** `http://127.0.0.1:8000/health`
- **Swagger API Docs:** `http://127.0.0.1:8000/docs`
- **Run Automated Tests (11 Tests Pass):** `pytest`

(Note: `.env` file-ல `AI_MODE=demo` மற்றும் `IMAGE_PROVIDER=placeholder` இருந்தா zero API key-ல பக்காவா ஓடும்!)

4. Top 15 Viva Questions & Answers (Examiner கண்டிப்பா கேக்க சான்ஸ் இருக்குற கேள்விகள்!)

Q1. Tell me about your project in simple words.

புரிஞ்சுக்க: உங்க ப்ராஜெக்ட் என்ன பண்ணுதுன்னு 2 வரியில சொல்லணும்.
Viva Answer: "Sir/Ma'am, ComicCraft is a FastAPI web application that takes a short story idea from the user and automatically creates a 3 to 8 panel illustrated comic book using Google Gemini AI, previews it on the web, and exports it as a downloadable A4 PDF."

Q2. Why did you use a Two-Stage Gemini Workflow instead of asking AI in a single prompt? [Star Question!]

புரிஞ்சுக்க: ஒரே வாட்டி AI கிட்ட கேட்டா கதையும் குழம்பும், character தோற்றமும் மாறிடும். அதனால ரெண்டு stage!
Viva Answer: "In a single prompt, LLMs often lose story pacing or change character appearance across panels. So in Stage 1 (`gemini-3.5-flash-lite`), we generate a structured JSON Outline and a **Character Bible**. In Stage 2 (`gemini-3.5-flash`), we pass that approved outline to generate panel-wise captions, narration, dialogue, and consistent image prompts."

Q3. How do you maintain Character Consistency across all comic panels?

புரிஞ்சுக்க: முதல் panel-ல ஒரு ட்ரெஸ், அடுத்த panel-ல வேற ட்ரெஸ் வராம எப்படி தடுக்குறீங்க?
Viva Answer: "During Stage 1, Gemini creates a `character_bible` containing fixed visual traits like face, hairstyle, outfit, and colors. This `character_bible` is repeated in every panel's image prompt, and we also pass negative prompts banning inconsistent costumes."

Q4. How do you make sure Gemini's output doesn't break your Python code?

புரிஞ்சுக்க: சாதாரணமா AI text குடுத்தா அத split பண்ணும்போது error வரும். நாம என்ன பண்றோம்?
Viva Answer: "We do not split free-form text. We request Gemini output with `response_mime_type='application/json'` and validate the JSON strictly using **Pydantic v2** models (`ComicOutline` and `ComicScript`), which verify exact panel counts and sequential panel numbering."

Q5. Why did you choose FastAPI instead of Flask or Django?

Viva Answer: "FastAPI is lightweight, high-performance, has built-in **Pydantic** request/response validation, automatically generates **Swagger OpenAPI documentation** at `/docs`, and provides `run_in_threadpool` to handle blocking AI and PDF tasks cleanly without freezing the async event loop."

Q6. Which Database did you use in this project? [Trick Question!]

புரிஞ்சுக்க: Examiner வேணும்னே "MySQL யூஸ் பண்ணியா, MongoDB யூஸ் பண்ணியா?"ன்னு கேப்பாங்க. மாட்டிக்காதீங்க!
Viva Answer: "We did not use an external SQL/NoSQL database. Instead, `ComicRepository` uses **UUID-isolated Local Filesystem Storage** (`storage/comics/<uuid>/`). Each comic folder stores PNG panels, `comic.pdf`, and an atomically written `comic.json` manifest. This keeps the app zero-dependency and portable."

Q7. How do you prevent security issues like Path Traversal when opening /comics/{comic_id}?

Viva Answer: "Every `comic_id` is generated as a 32-character lowercase hexadecimal UUID4, and `ComicRepository` strictly validates it against the regex `^[a-f0-9]{32}$` before accessing disk paths, preventing path traversal (`../`) attacks."

Q8. What Image Generation models/providers are supported?

Viva Answer: "We implemented a pluggable `ImageProvider` interface with three modes:
1. **Placeholder (`Pillow`)**: Fast, deterministic offline panel artwork (no API key/GPU needed).
2. **Hugging Face (`InferenceClient`)**: Cloud text-to-image using `stable-diffusion-xl-base-1.0` (SDXL).
3. **Diffusers**: Local Stable Diffusion (`v1.5`) loaded lazily when GPU/local inference is chosen."

Q9. What happens if Gemini API fails, hits rate-limit (429), or there is no internet? [Star Question!]

புரிஞ்சுக்க: API fail ஆனாலும் நம்ம ஆப் நிக்காது!
Viva Answer: "We built multi-level resilience:
1. For transient errors like `429` or `503`, it retries with **exponential backoff and jitter**.
2. For `400/404` model-not-found errors, it immediately switches to configured fallback Gemini models without wasting retries.
3. If all live calls fail and `ALLOW_AI_FALLBACK=true`, it automatically switches to the **Offline Demo Story Writer** and **Placeholder Image Provider**, completes the comic, and shows a transparent fallback warning banner to the user."

Q10. How are images reproducible (same prompt = same image seed)?

Viva Answer: "For each panel, we compute a deterministic integer seed using **SHA-256** hash of `comic_id + panel_number + image_prompt` (`sha256(...)[:8]`), ensuring consistent, reproducible generation per panel."

Q11. How is the PDF generated? Why did you bundle DejaVu fonts?

Viva Answer: "We use the **`fpdf2`** library in `PDFService` to create an A4 document with a styled dark cover page followed by one page per panel (preserving image aspect ratio inside a 180×145 mm box). We bundled **DejaVu Sans TTF** fonts so Unicode characters like smart quotes, em-dashes, and bullets render identically across Windows, Linux, and Android without encoding errors."

Q12. What is Uvicorn and Pydantic?

Viva Answer:
• **Uvicorn:** "A lightning-fast ASGI (Asynchronous Server Gateway Interface) web server used to run the FastAPI application."
• **Pydantic:** "A Python data validation library using type hints to validate user form inputs, `.env`

settings, and AI JSON outputs."

Q13. How are API Keys protected in your code?

Viva Answer: "API keys (`GEMINI_API_KEY`, `HF_TOKEN`) are never hardcoded. They are loaded from `.env` (which is git-ignored) using `pydantic-settings` and wrapped in `SecretStr` so they never leak in logs or `/health` endpoints."

Q14. How did you test the application?

Viva Answer: "We wrote an automated **`pytest`** test suite with **11 test cases** achieving ~80% code coverage. Tests use offline demo/placeholder providers and mock clients to test health endpoints, web form creation, JSON API, PDF export, and Gemini retry/model-switch logic without requiring internet or paid keys."

Q15. What are the Limitations and Future Enhancements of your project?

Viva Answer: "Currently, comic generation runs synchronously inside the HTTP request, uses local disk storage without user login, and supports 3–8 panels in English. As future enhancements, we can add Celery/Redis background task queues with real-time progress bars, user authentication with cloud storage (AWS S3 / PostgreSQL), single-panel editing/regeneration, multi-panel comic grid layouts, and regional language support like Tamil."

Examiner முன்னாடி கெத்தா பேச **4 Magic Keywords (**இத சொன்னாலே **Impress** ஆகிடுவாங்க**!):**

1. **"Two-Stage Gemini JSON Pipeline"** (Outline first, Script second)
2. **"Character Bible"** (for visual consistency across panels)
3. **"Strict Pydantic Schema Validation"** (no string splitting bugs)
4. **"Graceful Offline Fallback Architecture"** (works even without API keys or internet)